EVALUACIÓN

Pregunta 1: API y REST

¿Qué es una API y qué significa que sea REST?
Explica qué papel tienen las peticiones y las respuestas HTTP entre el cliente (navegador, Swagger o curl) y el servidor FastAPI.

Una API es una interfaz que permite que dos programas se comuniquen entre sí. REST es un estilo de arquitectura que utiliza métodos HTTP para intercambiar información. El cliente, como Swagger, curl o un navegador, envía peticiones al servidor FastAPI, y este las procesa y devuelve una respuesta con los datos o un mensaje de error.

Pregunta 2: Endpoints y métodos HTTP

¿Qué es un endpoint? Indica qué método HTTP y qué ruta usa cada operación de la API de tareas (crear, listar, obtener una, completar y borrar) y qué hace cada una.

Un endpoint es una dirección de la API que permite realizar una operación concreta.

- Crear: POST /tareas/ – Crea una nueva tarea.
- Listar: GET /tareas/ – Muestra todas las tareas.
- Obtener una: GET /tareas/{tarea_id} – Busca una tarea por su ID.
- Completar: PUT /tareas/{tarea_id}/completar – Marca una tarea como completada.
- Borrar: DELETE /tareas/{tarea_id} – Elimina una tarea.

Pregunta 3: Parámetros de ruta y de consulta

¿Qué diferencia hay entre un parámetro de ruta y un parámetro de consulta (query)? Pon un ejemplo de cada uno en la API de tareas (tarea_id y completada) y explica cómo detecta FastAPI cuál es cuál.

Un parámetro de ruta forma parte de la dirección URL y sirve para identificar un recurso. Por ejemplo, tarea_id en /tareas/2.
Un parámetro de consulta aparece después del símbolo ? y sirve para filtrar información. Por ejemplo, /tareas/?completada=true.
FastAPI distingue ambos automáticamente según dónde se declaran en la ruta y en la función.

Pregunta 4: Esquemas con Pydantic

¿Para qué sirven los esquemas de schemas.py? Explica la diferencia entre TareaCrear y TareaLeer, qué hace response_model y qué recibe el cliente si envía un cuerpo que no cumple el esquema.

Los esquemas de schemas.py sirven para validar y organizar los datos que recibe y devuelve la API.

- TareaCrear: Define los datos necesarios para crear una tarea.
- TareaLeer: Define los datos que se devuelven al consultar una tarea.

response_model indica el formato de la respuesta que recibirá el cliente. Si se envían datos incorrectos, FastAPI devuelve automáticamente un error 422.

Pregunta 5: Códigos de estado HTTP

¿Qué significan los códigos 201, 204, 404 y 422? Indica en qué endpoint de la API se devuelve cada uno y quién lo genera (tú con HTTPException, o FastAPI de forma automática).

- 201: Indica que se ha creado un recurso correctamente. Lo devuelve el endpoint de creación.
- 204: Indica que la operación se ha realizado correctamente, pero no devuelve contenido. Se utiliza al borrar una tarea.
- 404: Indica que el recurso solicitado no existe. Se genera mediante HTTPException.
- 422: Indica que los datos enviados no cumplen las condiciones necesarias. Lo genera FastAPI automáticamente al fallar la validación.

Pregunta 6: SQLite y el módulo sqlite3

¿Qué es SQLite y en qué se diferencia de un servidor de base de datos como PostgreSQL? ¿Dónde se guardan los datos en la Parte 1 y para qué se usa el módulo sqlite3 de Python?

SQLite es una base de datos que guarda la información en un archivo, sin necesitar un servidor independiente. A diferencia de PostgreSQL, no requiere un servicio de base de datos funcionando por separado.
En la Parte 1, los datos se guardan en un archivo local. El módulo sqlite3 de Python permite conectarse a esa base de datos, ejecutar consultas y gestionar la información.

Pregunta 7: La conexión a la base de datos

En la función get_connection(), ¿qué hacen check_same_thread=False y row_factory = sqlite3.Row? ¿Por qué son necesarios en una API con FastAPI?

En get_connection() se utilizan dos opciones importantes:

- check_same_thread=False: Permite utilizar la conexión desde diferentes hilos.
- row_factory = sqlite3.Row: Permite acceder a los datos de las filas utilizando el nombre de las columnas.

Son útiles para facilitar el trabajo con la base de datos dentro de FastAPI.

Pregunta 8: Dependencias con Depends

¿Para qué sirve Depends(get_db)? Explica por qué se abre una conexión por petición, qué papel tiene yield y por qué la conexión se cierra en el bloque finally.

Depends(get_db) sirve para proporcionar una conexión a la base de datos en los endpoints que la necesitan.
Se abre una conexión por petición para gestionar las consultas de forma independiente.
yield entrega la conexión al endpoint y permite continuar después con la limpieza.
Finalmente, el bloque finally cierra la conexión para evitar dejar recursos abiertos.

Pregunta 9: Consultas parametrizadas

¿Por qué se escribe WHERE id = ? y se pasan los valores en una tupla, en lugar de construir la consulta concatenando texto? Explica el riesgo de la inyección SQL.

Se utiliza WHERE id = ? para introducir los valores de forma segura mediante parámetros, en lugar de concatenar texto directamente en la consulta.
Esto evita la inyección SQL, un ataque en el que alguien introduce código malicioso para modificar o acceder a información de la base de datos.

Pregunta 10: Confirmar cambios y detectar errores

¿Por qué hace falta db.commit() después de un INSERT, un UPDATE o un DELETE? ¿Qué indica cursor.rowcount y cómo se usa para devolver un 404 cuando la tarea no existe?

db.commit() sirve para guardar definitivamente los cambios realizados mediante INSERT, UPDATE o DELETE.
cursor.rowcount indica cuántas filas han sido afectadas por una consulta. Si devuelve 0, significa que no se ha encontrado ninguna tarea para modificar o eliminar. En ese caso, se puede utilizar HTTPException con código 404 para informar de que la tarea no existe.